

ThreatLens

Security Assessment Platform

World Monitor - Security Assessment Report

CONFIDENTIAL - FOR AUTHORIZED RECIPIENTS ONLY

Project:	World Monitor
Target:	https://github.com/koala73/worldmonitor
Report Date:	September 17, 2026
Methodology:	Static Analysis . Dynamic Testing . Manual Validation
Prepared By:	ThreatLens Automated Scanner v1.0
Classification:	Confidential

1. EXECUTIVE SUMMARY

Overall Security Posture: MEDIUM RISK

5

TOTAL FINDINGS

0

CRITICAL

0

HIGH

1

MEDIUM

3

LOW

This automated security assessment identified 5 potential security issue(s) across 3 security category/categories. Of these, 5 remain open and 0 have been resolved. Critical and High severity findings should be addressed immediately.

Assessment Date: September 17, 2026

Target:

2. SEVERITY BREAKDOWN

Severity	Count	% of Total	Status
CRITICAL	0	0%	-
HIGH	0	0%	-
MEDIUM	1	20%	CONFIRMED
LOW	3	60%	REJECTED
INFO	1	20%	REJECTED

3. ASSESSMENT SCOPE & TARGET

Project Name: World Monitor

Description:

4. METHODOLOGY

ThreatLens performs a multi-phase security assessment combining static source-code analysis, dynamic application testing against authorized local targets, and automated validation. All testing is conducted in a non-destructive manner against explicitly authorized targets only. No testing is performed against third-party infrastructure, production systems outside the defined scope, or without explicit authorization.

Static Analysis:

Source code, configuration files, dependency manifests, and secret patterns are scanned without executing the application.

Dynamic Testing:

HTTP security headers, API endpoints, XSS reflection, and injection vectors are tested against a local authorized instance.

Validation:

Each finding is evaluated for confidence (CONFIRMED / LIKELY / POSSIBLE / FALSE_POSITIVE) using scanner-specific validation logic.

Evidence Collection:	Request/response pairs, code snippets, and configuration excerpts are captured as structured evidence.
Remediation Tracking:	Findings progress through a defined lifecycle: DETECTED -> CONFIRMED -> REMEDIATION -> RETESTING -> RESOLVED.

5. FINDINGS

F01 Potential: MCP Proxy DNS Rebinding Race (TOCTOU) on Vercel

MEDIUM

POSS

Confirmed

Location: api/mcp-proxy.ts:329

Description

POTENTIAL (architectural limitation, accepted P2). api/mcp-proxy.ts (Vercel Edge Function) performs two DNS pre-resolution checks (assertServerUrlSafe at line 329, revalidateBeforeFetch at line 368) before calling fetch(). A TOCTOU race window exists between the second DNS check and fetch()'s own DNS resolution -- if a DNS record is flipped between these two points, fetch() connects to a different IP than the one that was validated. A safe local PoC (mcp_toctou_poc.mjs) confirms the race window: DNS lookups 1-2 return a safe IP (203.0.113.1), while lookup 3 (simulating fetch()'s own resolution) returns 169.254.169.254. Significant mitigations in place: (1) DENIED_FORWARD_HEADERS strips GCP (Metadata-Flavor: Google), Azure (Metadata: true), and AWS IMDSv2 (X-aws-ec2-metadata-token) headers -- cloud metadata endpoints requiring auth headers cannot be exploited via this race; (2) mcp-proxy.ts exports config = { runtime: "edge" } -- this runs on Vercel Edge (Cloudflare Workers V8 isolates), which cannot reach 169.254.169.254 or RFC-1918 ranges; (3) Pro API key (paid subscription) is required to reach this code path; (4) Two DNS pre-checks narrow the race window significantly; (5) Code comment at line 360 explicitly acknowledges and accepts this limitation (issue #5061, P2). Practical exploitability is very low given the Cloudflare Workers runtime and metadata header stripping.

Remediation

Short-term (Edge runtime): no complete fix available -- Vercel Edge Functions lack Node.js socket-level hooks to pin connections. The existing DENIED_FORWARD_HEADERS defense mitigates the primary SSRF payload. Ensure DENIED_FORWARD_HEADERS stays current as new cloud providers add metadata endpoints. Long-term (issue #5061): migrate sensitive endpoints to Vercel Serverless (Node.js) where http.request() socket-level IP pinning IS available -- resolve DNS, assert, then pass the IP directly to http.request({host: validatedIp, ...}) rather than the hostname.

Evidence

[CODE_SNIPPET] Safe local PoC confirms TOCTOU race window (DNS lookups 1-2 pass, lookup 3 rebinds)

PoC file: scratchpad/mcp_toctou_poc.mjs. Test environment: Node.js 20, localhost only, no external network calls. PoC output: ... (truncated)

F02 Dependency: image-size <=2.0.2 - DoS via Infinite Loop in Image

LOW

FALS

Rejected

Location: package-lock.json:1

Description

FALSE POSITIVE (build-time tooling, not runtime). image-size <=2.0.2 (GHSA-w3rx-r6r6-pgpr, GHSA-5p2g-fcmc-qvqq) is present in the lockfile as a transitive dep: @loaders.gl/textures -> texture-compressor -> image-size. Verification confirms the vulnerable code path is UNREACHABLE at runtime: (1) src/shims/child-process-proxy.ts is a browser shim that throws on ChildProcessProxy.start() -- texture-compressor cannot be spawned from the Vite/browser build or Vercel Edge Functions; (2) grep of all TypeScript/JavaScript source finds zero imports of @loaders.gl/textures, ScenegraphLayer, SimpleMeshLayer, or any GLTF loading path; (3) DeckGLMap.ts imports only 2D layer types (GeoJsonLayer, ArcLayer, etc.) -- no 3D mesh layers; (4) texture-compressor.main = "dist/cli/lib/index.js" -- it is a CLI tool, not an importable library. The dependency exists in the tree and should be updated for hygiene, but it cannot be triggered by any HTTP request to the application.

Remediation

Hygiene fix only (not security-critical): add a package.json override to pin image-size to >=3.0.0 to suppress npm audit warnings: {"overrides": {"image-size": ">=3.0.0"}}. No code change is required because the vulnerable path is not reachable at runtime.

Evidence

[DEPENDENCY_RECORD] FALSE POSITIVE: image-size in lockfile but unreachable at runtime (ChildProcessProxy shim)

```
Verification steps: (1) grep -rn "@loaders.gl/textures" src/ api/ ? 0 results. (2) grep -rn "MeshLayer|ScenegraphLayer|S
... (truncated)
```

F03 Dependency: stream-json - O(depth^2) DoS via Deeply Nested JSON

LOW

FALSE

Rejected

Location: package-lock.json:1

Description

FALSE POSITIVE. npm audit flagged stream-json via the chain @solana/web3.js -> jayson -> stream-json. All three packages are marked peer:true in package-lock.json ? peer deps are only installed when explicitly declared by the app. No World Monitor app code imports stream-json or jayson. The vulnerable O(depth^2) filter methods are never called in any app code path. This is a false positive from npm audit lockfile-based analysis.

Remediation

1. Upgrade stream-json to patched version. 2. Enforce JSON depth limits (max 20) before stream-json pipelines. 3. Audit call sites for user-controlled JSON input.

Evidence

[DEPENDENCY_RECORD] npm audit - stream-json O(depth^2) DoS (GHSA-528h-pc64-c93x)

```
npm audit:
stream-json Severity: moderate
Event loop blocked seconds-to-minutes on crafted deeply-nested JSON
https://github.com/advisories/GHSA-528h-pc64-c93x
```

F04 Dependency: uuid v3/v5/v6 - Missing Buffer Bounds Check on Custom buf argument

LOW

FALSE

Rejected

Location: package-lock.json:1

Description

FALSE POSITIVE. uuid v8.3.2 is in the lockfile and would be installed, but GHSA-w5hq-g745-h8pq only affects uuid.v3(), uuid.v5(), and uuid.v6() when called with a custom buf argument of insufficient length. A complete search of all TypeScript and JavaScript source in World Monitor found zero calls to uuid.v3(), .v5(), or .v6() with any buf argument. Only standard v4() UUID generation is used. The vulnerable code path is not exercised by the application.

Remediation

1. Upgrade uuid to patched version. 2. Audit uuid.v3/v5/v6 call sites for external buf arguments accepted from user input.

Evidence

[DEPENDENCY_RECORD] npm audit - uuid missing bounds check (GHSA-w5hq-g745-h8pq)

```
npm audit:
uuid Severity: moderate
Missing buffer bounds check in uuid.v3/v5/v6 when buf is provided
https://github.com/advisories/GHSA-w5hq-g745-h8pq
```

F05 Informational: GitHub Actions CI Uses Default PostgreSQL Credentials (postg





Location: `.github/workflows/test.yml:553`

Description

FALSE POSITIVE (secrets scanner). The workflow file contains `UMAMI_TEST_DATABASE_URL=postgresql://postgres:postgres@127.0.0.1:5432/umami_integration`. The string `postgres:postgres` is the official PostgreSQL Docker image factory default ? not a real secret. This is a CI-only ephemeral database bound to `127.0.0.1`, destroyed after each CI run, with no access to any production system. The credential carries no secret value and matches the known-safe CI test pattern.

Remediation

- 1. Generate random credentials via GitHub Actions secrets.
- 2. Use `${{ secrets.CI_DB_PASSWORD }}` for ephemeral test databases.
- 3. Add comment documenting CI-only scope if static credentials are retained.

Evidence

[CONFIG_VALUE] Hardcoded postgres:postgres in `.github/workflows/test.yml:553`

```
.github/workflows/test.yml lines 551-558:
  UMAMI_TEST_DATABASE_URL: postgresql://postgres:postgres@127.0.0.1:5432/umami_integration
  (used in two consecutive test steps)
Note: postgres:postgres are the official PostgreSQL Docker image default credentials.
CI-only ephemeral test database, not a production credential.
```

6. SECURITY POSTURE & CONCLUSION

Rating: MEDIUM RISK

Assessment conducted by ThreatLens Automated Security Platform

No significant security issues were detected. Maintain regular assessments.

NOTE: This report was generated by ThreatLens automated scanning. Results should be reviewed by a qualified security professional before making security decisions. Automated tools cannot detect every vulnerability type.